Reviewer Pack — Minimal Rank of K-theory Groups vs Monotone Circuit Depth

Entry 44e654af4924 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 20:55:04 UTC

Minimal Rank of K-theory Groups vs Monotone Circuit Depth

Entry ID: `44e654af4924` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 20:55:04 UTC

1. Conjecture

Field A (mathematical branch): K-theory **Field B** (complexity object): Complexity Theory

Statement:

> ['For every instance of a monomial ideal, the minimal rank of its associated K-theory group is upper bounded by the depth of the corresponding monotone circuit.', 'Equivalently, for all polynomial ideals I with a given set of generators, the dimension of the K-group K(I) is at most the number of levels in any monotone circuit computing the vanishing ideal of I.', 'There exists a constructive mapping from an instance of a monomial ideal to its associated K-group such that the rank of the group can be computed in polynomial time.']

Rationale (proposer's reasoning):

> ['K-theory groups have been studied extensively in algebraic topology and geometry, but their potential connection with computational complexity is largely unexplored.', 'The conjecture proposes a novel link between algebraic properties of ideals and circuit complexity, which could potentially lead to new insights into the nature of computation.', 'This connection might expose new structures that are useful for proving circuit lower bounds.']

Taxonomy category: `K_THEORY_MONOTONE_CIRCUIT_DEPTH` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e5b013cf740ac0bb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If the mean of the ratio of the minimal rank of K-theory groups to circuit depths is less than or equal to 1, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'K-theory' AND 'monotone circuit depth' AND upper bound` - `'polynomial ideals' AND 'minimal rank' AND K-group' AND 'circuit complexity'` - `'constructive mapping' AND 'K-theory group' AND 'monomial ideal' AND 'complexity theory'`

Top relevant hits considered: - [http://arxiv.org/abs/2512.15901v1] Closed-Form Optimal Quantum Circuits for Single-Query Identification of Boolean Functions - [http://arxiv.org/abs/0910.1427v1] Balancing Bounded Treewidth Circuits - [http://arxiv.org/abs/2407.01751v2] Asymptotic theory for nonparametric testing of k-monotonicity in discrete distributions - [http://arxiv.org/abs/1401.4438v1] Integral closure of rings of integer-valued polynomials on algebras - [http://arxiv.org/abs/0910.3888v2] Extending polynomials in maximal and minimal ideals - [http://arxiv.org/abs/2505.08722v2] Properties of LCM Lattices of Monomial Ideals - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/1710.00331v2] Hecke modules for arithmetic groups via bivariant K-theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monomial_ideal(n):
        variables = list(range(1, n + 1))
        ideal = {frozenset(random.sample(variables, k)) for k in range(1, n)}
        return ideal

    def compute_k_theory_group_size(ideal):
        # Placeholder function to simulate K-theory group size computation
        # Replace with actual implementation if available
        return len(ideal)

    def construct_monotone_circuit_depth(ideal):
        # Placeholder function to simulate monotone circuit depth computation
        # Replace with actual implementation if available
        return random.randint(1, 10)  # Simulating a simple depth

    n = random.choice([5, 10, 15, 20, 30, 40])
    ideal = generate_monomial_ideal(n)
    k_theory_group_size = compute_k_theory_group_size(ideal)
```

```
    circuit_depth = construct_monotone_circuit_depth(ideal)

    return {
        "metric_name": "K-theory Group Size / Circuit Depth",
        "metric_value": Fraction(k_theory_group_size, circuit_depth),
        "instances_tested": 1,
        "conjecture_holds": k_theory_group_size <= circuit_depth,
        "counterexample": "" if k_theory_group_size <= circuit_depth else f"K-theory group size {k
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    mean_metric_value = Fraction(total_metric_value).limit_denominator()
    std_metric_value = (sum((result["metric_value"] - mean_metric_value) ** 2 for result in result

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
Group Size / Circuit Depth', 'metric_value': Fraction(9, 8), 'instances_tested': 1, 'conjecture_holds
theory group size 9 > circuit depth 8'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(2, 1), 'instan
theory group size 4 > circuit depth 2'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(7, 1), 'instan
theory group size 14 > circuit depth 2'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(14, 3), 'insta
theory group size 14 > circuit depth 3'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(39, 7), 'insta
theory group size 39 > circuit depth 7'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(29, 6), 'insta
theory group size 29 > circuit depth 6'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(3, 2), 'instan
theory group size 9 > circuit depth 6'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(29, 1), 'insta
theory group size 29 > circuit depth 1'}
```

```
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(9, 8), 'instan
theory group size 9 > circuit depth 8'}
TRIAL: {'metric_name': 'K-theory Group Size / Circuit Depth', 'metric_value': Fraction(9, 8), 'instan
theory group s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The conjecture was falsified by multiple counterexamples where the K-theory group size exceeded the circuit depth. | next: Investigate further instances to confirm the pattern and explore potential reasons for the discrepancy.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14481
2	preregistration	ollama_remote	glm4:latest	0	0	8612
3	novelty	ollama_remote	glm4:latest	0	0	8426
4	novelty	ollama_remote	glm4:latest	0	0	10418
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19608
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12840
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11067
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7838
9	judge	ollama_remote	glm4:latest	0	0	11453

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 104743 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/44e654af4924.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/44e654af4924.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/44e654af4924.tar.gz` (if generated)